Milestone 6: Next.js — Pages, Routing & Beyond

🔹 Next.js Foundation and Routing

What Is Next.js? Why Choose Next.js?

Next.js is a React framework — it's built ON TOP of React, adding essential production features out of the box: file-based routing, server-side rendering, image optimization, and more, so you don't have to configure and wire these together yourself.

💡 Analogy: If React is a box of high-quality engine parts, Next.js is the fully assembled car — the engine (React) is still what powers everything, but the chassis, wheels, and dashboard (routing, rendering, optimization) are already built in and ready to drive.

Creating a Next.js App and Understanding Project Structure

A new Next.js project is created with a single CLI command, which scaffolds a working app with sensible defaults. The App Router's key folders/files: app/ (every route lives here), app/page.tsx (the homepage), app/layout.tsx (the root layout wrapping every page), public/ (static files like images), and next.config.js (project configuration).

```
npx create-next-app@latest my-app
cd my-app
npm run dev

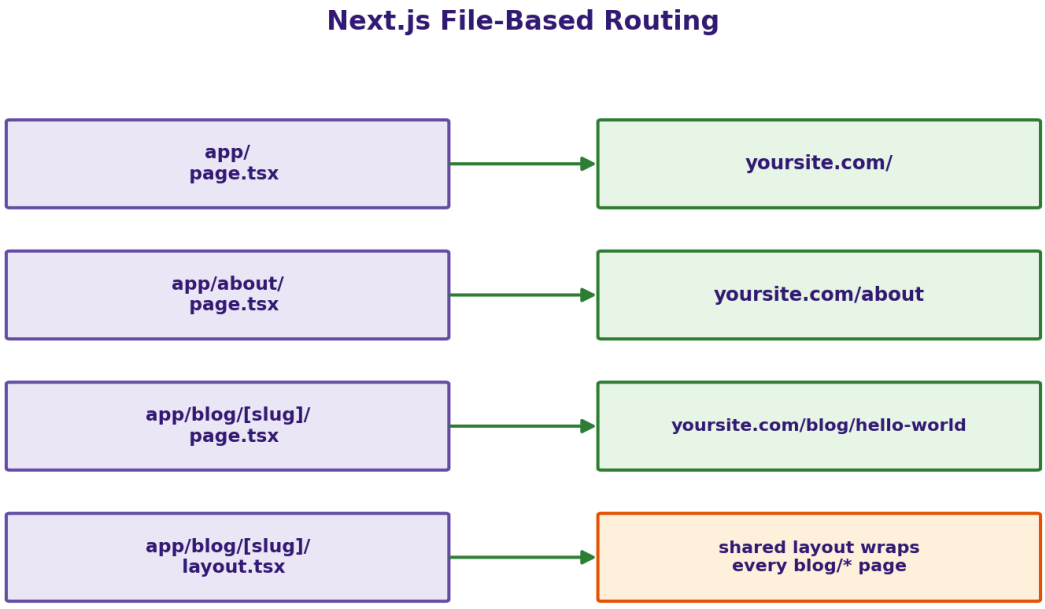
// Key structure:
// app/
//   layout.tsx   ← root layout (wraps every page)
//   page.tsx     ← the homepage ("/")
//   globals.css  ← global styles
// public/        ← static assets (images, icons)
```

Create Pages, Routing in Next.js, and Navigation

Next.js uses file-based routing: the folder structure inside app/ directly maps to your site's URLs — no manual route configuration needed. Each route folder needs a page.tsx file to actually render something at that URL. Navigation between pages uses the built-in <Link> component instead of a plain <a> tag, which enables fast client-side transitions without a full page reload.

💡 Analogy: File-based routing is like a filing cabinet where the folder names ARE the

index — there's no separate table of contents (route config) to maintain; the folder structure itself tells Next.js what URL maps to what page.




```
// app/about/page.tsx → renders at yoursite.com/about
export default function AboutPage() {
  return <h1>About Us</h1>;
}

// Navigating between pages
import Link from "next/link";

function Navbar() {
  return (
    <nav>
      <Link href="/">Home</Link>
      <Link href="/about">About</Link>
    </nav>
  );
}
```

Install DaisyUI and Navigation in Next.js

DaisyUI (the Tailwind component library from Milestone 5) installs the same way in a Next.js project, giving pre-styled components like navbar, btn, and drawer to speed up building the navigation bar and overall layout.

```
npm install -D daisyui
```

```
// tailwind.config.js
module.exports = {
  plugins: [require("daisyui")]
};

// Using a daisyUI navbar component:
// <div className="navbar bg-base-100">
//   <a className="btn btn-ghost text-xl">MySite</a>
// </div>
```

Dynamic Routing in Next.js and Dynamic Route Segments

A dynamic route uses square brackets in the folder name — like [id] or [slug] — to match many different URLs with a single page template, where the bracketed part becomes a parameter your component can read. This is essential for pages like product details or blog posts, where you can't (and shouldn't) create a separate file for every single item.

💡 Analogy: A dynamic route is like a fill-in-the-blank form letter — one single template ([slug]/page.tsx) serves thousands of different URLs, just swapping in a different name each time.

```
// app/blog/[slug]/page.tsx
interface PageProps {
  params: { slug: string };
}

export default function BlogPost({ params }: PageProps) {
  return <h1>Reading post: {params.slug}</h1>;
}
// Matches: /blog/hello-world, /blog/my-first-post, etc.
```

Creating Multiple Layouts in Next.js

A layout.tsx file wraps every page inside its folder (and all nested subfolders) with shared UI — like a navbar or sidebar — without that UI being re-rendered on every navigation. Different sections of a site can have their own distinct layout by placing a layout.tsx deeper in the folder structure, nested inside the root layout.

💡 Analogy: Layouts are like nested picture frames — the root layout is the outer frame around the whole wall, and a nested layout is a smaller frame around just one picture, both visible at the same time.

```
// app/layout.tsx — root layout, wraps EVERY page
export default function RootLayout({ children }: { children: React.ReactNode
}) {
  return (
```

```
    <html lang="en">
      <body>
        <Navbar />
        {children}
        <Footer />
      </body>
    </html>
  );
}

// app/dashboard/layout.tsx — wraps only /dashboard/* pages
export default function DashboardLayout({ children }: { children:
React.ReactNode }) {
  return (
    <div className="flex">
      <Sidebar />
      <main>{children}</main>
    </div>
  );
}
```

(Recap) Dynamic Routing and Data Loading in Next.js

Combining dynamic routes with data fetching is the core pattern for detail pages: read the dynamic parameter (like params.id), use it to fetch the specific matching data (from an API or database), and render the page once the data arrives — all of which can happen directly on the server before the page is even sent to the browser.

```
// app/products/[id]/page.tsx
async function getProduct(id: string) {
  const res = await fetch(`https://api.example.com/products/${id}`);
  return res.json();
}

export default async function ProductPage({ params }: { params: { id: string }
}) {
  const product = await getProduct(params.id);
  return <h1>{product.name}</h1>;
}
```

Image Optimization in Next.js

The built-in next/image component automatically optimizes images: it resizes them for the actual display size, serves modern formats like WebP when supported, and lazy-loads images that are off-screen — all without any extra configuration, unlike a plain <img> tag which always loads the full-size file.

💡 Analogy: next/image is like a smart photo lab that automatically prints the right size photo for whichever frame you're using, instead of handing you one giant print you have to manually resize yourself every time.

```
import Image from "next/image";

function ProductCard() {
  return (
    <Image
      src="/product.jpg"
      alt="Product photo"
      width={400}
      height={300}
      priority // loads immediately, skip lazy-loading (for above-the-fold
images)
    />
  );
}
```

SEO Metadata, Not Found, and Active Link

Next.js lets each page export a metadata object controlling its title, description, and social-sharing tags for search engines — far simpler than manually managing <head> tags. A not-found.tsx file customizes the 404 page shown when no route matches. 'Active link' styling highlights the current page in a navbar, typically using the usePathname hook to compare the current URL against each link.

```
// app/about/page.tsx
export const metadata = {
  title: "About Us | MySite",
  description: "Learn more about our company."
};

// app/not-found.tsx
export default function NotFound() {
  return <h1>404 — Page Not Found</h1>;
}

// Active link example (client component)
"use client";
import { usePathname } from "next/navigation";
import Link from "next/link";

function NavLink({ href, children }: { href: string; children: React.ReactNode
}) {
  const pathname = usePathname();
  const isActive = pathname === href;
  return <Link href={href} className={isActive ? "font-bold text-blue-600" :
```

```
""}>{children}</Link>;
}
```

Apply Google Font

Next.js has built-in, automatic optimization for Google Fonts via next/font/google — it downloads and self-hosts the font at build time (no external request to Google's servers at runtime), which improves both performance and privacy.

```
import { Inter } from "next/font/google";

const inter = Inter({ subsets: ["latin"] });

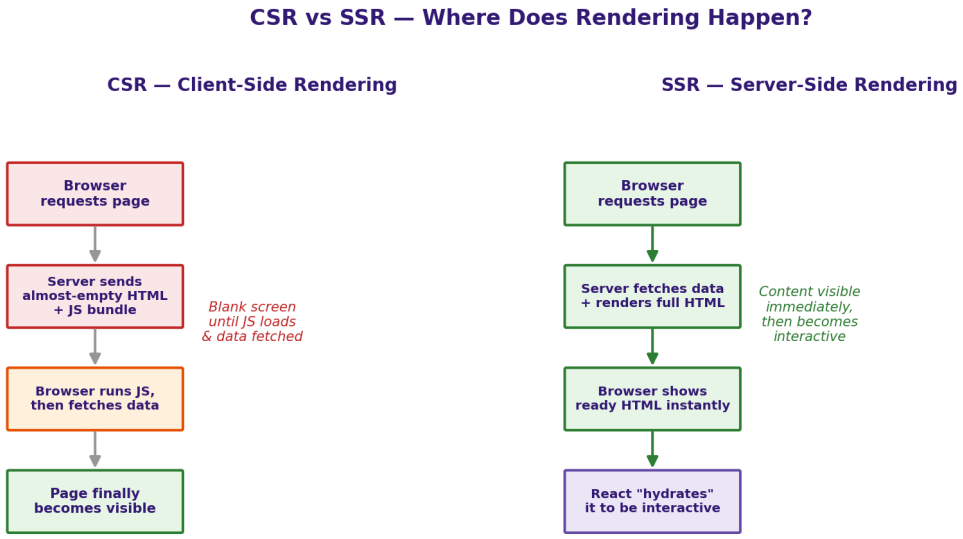
export default function RootLayout({ children }: { children: React.ReactNode
}) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
```

🔹 Next.js Rendering Basics

What Is Rendering? What Is CSR and SSR?

Rendering is the process of turning your component code and data into the actual HTML a browser displays. Client-Side Rendering (CSR) means that HTML is built in the user's browser, after JavaScript downloads and runs. Server-Side Rendering (SSR) means the HTML is built on the server first, then sent to the browser already complete.

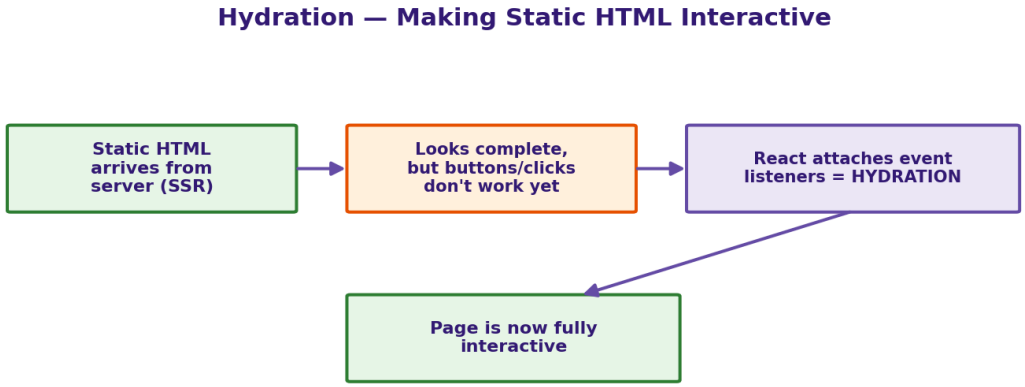
💡 Analogy: CSR is like a restaurant that hands you raw ingredients and a recipe card, and you cook the meal yourself at your table. SSR is like the kitchen cooking the meal fully before it ever reaches you — you just sit down to a plate that's already ready.



About Hydration, CSR vs SSR, and Next.js Rendering Techniques

Hydration is the step where React 'wakes up' server-rendered HTML by attaching event listeners and internal state to it in the browser, turning static markup into a fully interactive page. Next.js supports multiple rendering techniques (SSR, Static Site Generation, Incremental Static Regeneration, and CSR) and lets you choose the right one per page or even per component.

💡 Analogy: Hydration is like a mannequin in a shop window (static HTML) suddenly coming to life once its 'nervous system' (React's event listeners) gets connected — it looked complete before, but now it can actually respond to being touched.

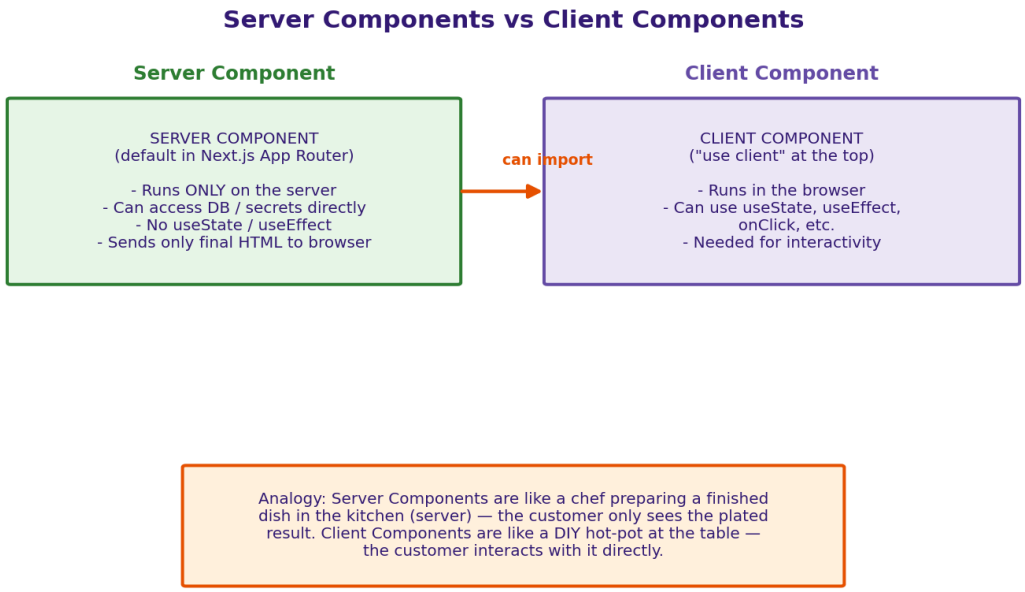


Analogy: hydration is like a mannequin (static HTML) coming to life once its 'nervous system' (React's event listeners) is connected.

Client Components vs Server Components

In the Next.js App Router, every component is a Server Component by default — it runs only on the server, can access databases/secrets directly, and sends only final HTML to the browser (smaller JS bundle). Adding "use client" at the very top of a file makes it a Client Component, which runs in the browser and can use interactive features like useState, useEffect, and onClick. A Server Component can import and render a Client Component, but not the other way around.

💡 Analogy: Server Components are like a chef preparing a finished dish entirely in the kitchen — the customer (browser) only receives the plated result. Client Components are like a DIY hot-pot at the table — the customer directly interacts with the components themselves.




```
// Server Component (default, no directive needed)
async function ProductList() {
  const products = await fetch("https://api.example.com/products").then(r ⇒
r.json());
  return <ul>{products.map((p: any) ⇒ <li key={p.id}>{p.name}</li>)}</ul>;
}

// Client Component — needs "use client" for interactivity
"use client";
import { useState } from "react";
```

```
function LikeButton() {
  const [liked, setLiked] = useState(false);
  return <button onClick={() ⇒ setLiked(!liked)}>{liked ? "❤️ Liked" : "🤍
Like"}</button>;
}
```

Loading Data Using Client Component

When data needs to be fetched in response to user interaction (like a search box typing), or when the data-fetching library requires browser-only features, you fetch inside a Client Component using the familiar useEffect + useState pattern — the same approach as plain React, just inside a file marked "use client".

```
"use client";
import { useState, useEffect } from "react";

function SearchResults({ query }: { query: string }) {
  const [results, setResults] = useState([]);

  useEffect(() ⇒ {
    fetch(`/api/search?q=${query}`)
      .then(res ⇒ res.json())
      .then(data ⇒ setResults(data));
  }, [query]);

  return <ul>{results.map((r: any) ⇒ <li key={r.id}>{r.name}</li>)}</ul>;
}
```

Recap: Data Fetching and Display Data

There are now two main data-fetching patterns in Next.js: fetch directly inside an async Server Component (runs on the server, no loading spinner needed since the HTML arrives already populated), or fetch inside a Client Component with useEffect (runs in the browser, needs a loading state). Choosing between them depends on whether the data is needed immediately for the page, or only in response to client-side interaction.

Create Dynamic Link to Display Food Details

Combining <Link> with a dynamic route lets you build a list page where each item links to its own detail page — each food item in a list renders a <Link href={`/food/${item.id}`}>, matching the [id]/page.tsx dynamic route.

```
// app/food/page.tsx (list)
import Link from "next/link";

function FoodList({ foods }: { foods: { id: string; name: string }[] }) {
  return (
    <ul>
```

```
      {foods.map(food => (
        <li key={food.id}>
          <Link href={`/food/${food.id}`}>{food.name}</Link>
        </li>
      ))}
    </ul>
  );
}
```

Use Loading, Handling Error, and Route-Specific Data Load

Next.js has special files that automatically wrap a route's content: loading.tsx shows a fallback UI (like a spinner) automatically while a Server Component's data is being fetched, and error.tsx catches runtime errors in that route segment and displays a fallback instead of crashing the whole app.

```
// app/food/[id]/loading.tsx — shown automatically while data loads
export default function Loading() {
  return <p>Loading food details ...</p>;
}

// app/food/[id]/error.tsx — shown automatically if something throws
"use client";
export default function Error({ error }: { error: Error }) {
  return <p>Something went wrong: {error.message}</p>;
}
```

🔹 Advanced Data Fetching & UI Integration

Recap: Data Fetching Flow in Next.js

The full picture: Server Components fetch data directly with async/await at render time; Next.js extends the native fetch() function with its own caching system on top, controlling whether that data is reused across requests or fetched fresh every time — this caching behavior is what Modules 35-4 through 35-6 cover in depth.

Fetching Data in Server Components

Because Server Components can be async functions directly, you simply await fetch() (or a database call) right inside the component body — no useEffect, no loading state management, since the component doesn't render until the data is ready.

```
async function BookList() {
  const res = await fetch("https://api.example.com/books");
  const books = await res.json();
```

```
  return <ul>{books.map((b: any) => <li key={b.id}>{b.title}</li>)}</ul>;
}
```

Set JSON Server with Temporary Data Server

json-server is a quick tool that turns a plain JSON file into a fully working fake REST API in seconds — perfect for practicing data fetching before a real backend exists.

```
npm install -g json-server

// db.json
// { "books": [{ "id": 1, "title": "Learning Next.js" }] }

json-server --watch db.json --port 3001
// Now GET http://localhost:3001/books returns the books array
```

Understanding force-cache and Cache Checking

By default, Next.js's extended fetch() caches the result indefinitely (equivalent to passing { cache: "force-cache" }), reusing it across requests instead of hitting the network again — great for data that rarely changes.

💡 Analogy: force-cache is like photocopying a document once and handing out copies from that stack forever, instead of reprinting the original every single time someone asks for it.

```
// Explicit force-cache (same as the default behavior)
const res = await fetch("https://api.example.com/books", {
  cache: "force-cache"
});
```

Dynamic Data Fetching: cache: 'no-store' and revalidate

{ cache: "no-store" } tells fetch to skip caching entirely and get fresh data on every single request — ideal for frequently changing data like live prices or a user's notifications. { next: { revalidate: N } } takes a middle path: cache the result, but automatically treat it as stale and refetch after N seconds.

💡 Analogy: no-store is like asking the kitchen to cook a brand-new meal for every single order, with zero reuse. revalidate(N) is like a buffet tray that gets refilled with a fresh batch every N minutes, rather than for every single customer.

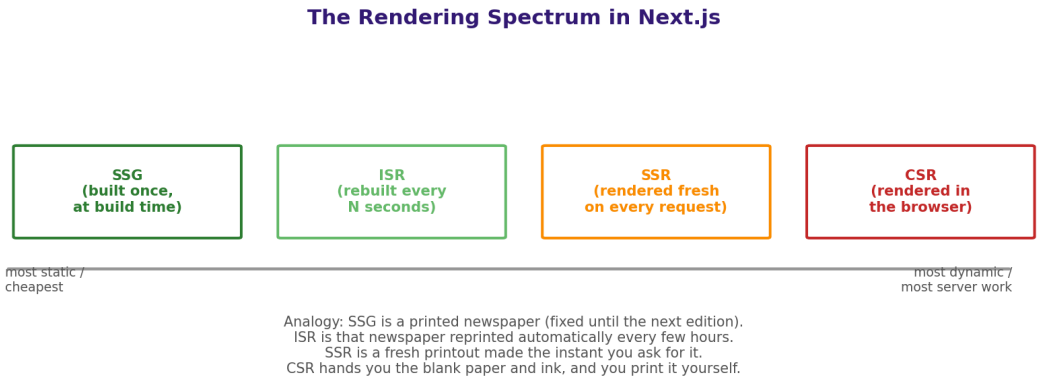
```
// Always fresh — no caching at all
const res = await fetch("https://api.example.com/live-price", {
  cache: "no-store"
});
```

```
// Cached, but refreshed automatically every 60 seconds
const res2 = await fetch("https://api.example.com/books", {
  next: { revalidate: 60 }
});
```

(Recap) ISR with Revalidate and SSG vs ISR

Static Site Generation (SSG) renders a page once, at build time, and serves that exact same HTML to every visitor until the next deployment — fastest possible response, but data can go stale. Incremental Static Regeneration (ISR) is SSG with a twist: using revalidate, Next.js automatically regenerates that static page in the background after the specified time, so content stays reasonably fresh without needing a full SSR request every time.

💡 Analogy: SSG is a newspaper printed once and never updated until the next edition. ISR is that same newspaper automatically reprinted every few hours with the latest news, while still being handed out as a pre-printed copy, not written fresh for each reader.



[Advanced] generateStaticParams (Dynamic Pages Optimization)

generateStaticParams tells Next.js, ahead of time, exactly which dynamic route values (like which product IDs) should be pre-rendered at build time as static pages — combining the performance of SSG with the flexibility of dynamic routes, instead of rendering every dynamic page on-demand.

```
// app/products/[id]/page.tsx
export async function generateStaticParams() {
  const products = await fetch("https://api.example.com/products").then(r =>
r.json());
  return products.map((p: any) => ({ id: p.id.toString() }));
```

```
}

export default function ProductPage({ params }: { params: { id: string } }) {
  // This page is now pre-built at compile time for every returned id
  return <h1>Product {params.id}</h1>;
}
```

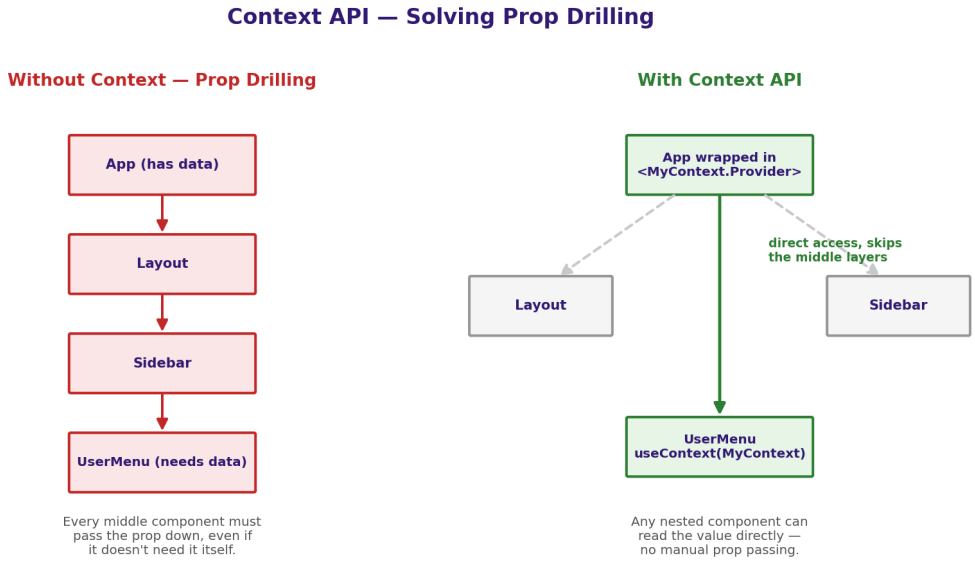
Check generateStaticParams

After adding generateStaticParams, running the production build (npm run build) shows which routes were pre-rendered as static pages (marked ● or similar in the build output) versus rendered dynamically on-demand — a useful way to verify the optimization is actually working as intended.

[React Concept] Explore Context API

The Context API solves 'prop drilling' — the awkward pattern of passing a prop down through several layers of components that don't need it themselves, just so a deeply nested component can finally receive it. Context lets you define a value once at a high level and read it directly from any descendant component, no matter how deeply nested, without passing it through every component in between.

💡 Analogy: Prop drilling is like passing a note hand-to-hand through a line of people who don't need to read it, just to get it to the last person. Context API is like a shared bulletin board (or a WiFi broadcast) — anyone in the building can walk up and read it directly, with no human chain required.



```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext<string>("light");

function App() {
  const [theme] = useState("dark");
  return (
    <ThemeContext.Provider value={theme}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <ThemedButton />; // no need to pass 'theme' through here
}

function ThemedButton() {
  const theme = useContext(ThemeContext); // read it directly
  return <button className={theme}>Click</button>;
}
```

[React Concept] Explore Custom Hook

A custom hook is simply a regular function (named starting with use) that packages up reusable stateful logic — like fetching data, or tracking window size — so that logic can be shared across multiple components without duplicating code. It can internally use other hooks (useState, useEffect, useContext) just like a component can.

💡 Analogy: A custom hook is like a reusable power tool — instead of each component rebuilding the exact same wiring from scratch, they just 'plug in' the shared tool and get the same capability every time.

```
import { useState, useEffect } from "react";

// Custom hook — reusable fetch logic
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(result => { setData(result); setLoading(false); });
  }, [url]);

  return { data, loading };
```

```
}

// Used in any component:
function BookList() {
  const { data: books, loading } = useFetch<Book[]>("/api/books");
  if (loading) return <p>Loading…</p>;
  return <ul>{books?.map(b => <li key={b.id}>{b.title}</li>)}</ul>;
}
```